

Week11_예습과제_권지원

Chapter 07. 군집화 (Clustering)

01. K-평균 알고리즘 이해

K-평균(K-Means) 개요

- 군집화(Clustering)에서 가장 일반적으로 사용되는 알고리즘
- 군집 중심점(centroid)이라는 특정 임의의 지점을 선택해 해당 중심에 가장 가까운 포인트들을 선택하는 군집화 기법
- 군집 중심점은 선택된 포인트의 평균 지점으로 이동하고, 이동된 중심점에서 다시 가까운 포인트를 선택하는 프로세스를 반복 수행
- 모든 데이터 포인트에서 더 이상 중심점의 이동이 없을 경우 반복을 멈추고 해당 중심점에 속하는 데이터 포인트들을 군집화

K-평균 동작 원리 (단계별)

```
flowchart LR
    A["① 2개의 군집 중심점 설정\\n(임의의 위치)"] --> B["② 각 데이터는 가장\\n가까운 중심점에 소속"]
    B --> C["③ 중심점에 할당된 데이터들의\\n평균 중심으로 중심점 이동"]
    C --> D["④ 각 데이터는 이동된\\n중심점 기준으로 재소속"]
    D --> E["⑤ 다시 평균 중심으로\\n중심점 이동"]
    E --> F{"중심점 소속\\n변경 없음?"}
    F -- "Yes" --> G["군집화 완료 ✅"]
    F -- "No" --> D
```



세부 단계 설명

1. **중심점 초기화** : 군집화 개수만큼 임의의 위치에 중심점 배치
 - 임의의 위치에 배치 시 반복 횟수 증가로 수행 시간 증가 → 실제로는 초기화 알고리즘(k-means++) 사용
2. **소속 결정** : 각 데이터는 가장 가까운 중심점에 소속
3. **중심점 이동** : 소속된 데이터들의 평균 위치로 중심점 이동
4. **재소속** : 이동된 중심점 기준으로 각 데이터의 소속 재결정
5. **반복** : 중심점 재이동 수행
6. **종료** : 중심점 이동 후 데이터의 소속 변경이 없으면 군집화 종료, 변경이 있으면 4단계부터 반복

K-평균의 장단점

장점

- 일반적인 군집화에서 가장 많이 활용되는 알고리즘
- 알고리즘이 쉽고 간결함

단점

- 거리 기반 알고리즘으로 속성의 개수가 매우 많을 경우 군집화 정확도 저하
 - 이를 위해 PCA로 차원 감소를 적용해야 할 수 있음
- 반복을 수행하는데 반복 횟수가 많을 경우 수행 시간이 매우 느려짐
- 몇 개의 군집(cluster)을 선택해야 할지 가이드하기 어려움

사이킷런 KMeans 클래스 소개

```
class sklearn.cluster.KMeans(
    n_clusters=8,
    init='k-means++',
    n_init=10,
    max_iter=300,
    tol=0.0001,
    precompute_distances='auto',
    verbose=0,
    random_state=None,
    copy_x=True,
    n_jobs=1,
```

```
algorithm='auto'
)
```

주요 초기화 파라미터

파라미터	설명
<code>n_clusters</code>	군집화할 개수, 즉 군집 중심점의 개수 (가장 중요한 파라미터)
<code>init</code>	초기 군집 중심점 좌표 설정 방식. 일반적으로 <code>k-means++</code> 사용
<code>max_iter</code>	최대 반복 횟수. 이 횟수 이전에 모든 데이터의 중심점 이동이 없으면 종료

주요 속성

속성	설명
<code>labels_</code>	각 데이터 포인트가 속한 군집 중심점 레이블
<code>cluster_centers_</code>	각 군집 중심점 좌표 (Shape: [군집 개수, 피쳐 개수]). 군집 중심점 좌표 시각화에 활용

- `fit(데이터 세트)` 또는 `fit_transform(데이터 세트)` 메서드를 이용해 수행

K-평균을 이용한 붓꽃 데이터 세트 군집화

데이터 로드

```
from sklearn.preprocessing import scale
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
%matplotlib inline

iris = load_iris()

# 편리한 데이터 핸들링을 위해 DataFrame으로 변환
irisDF = pd.DataFrame(data=iris.data,
                      columns=['sepal_length', 'sepal_width', 'petal_length', 'petal_width'])

irisDF.head(3)
```

출력 결과

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2

KMeans 군집화 수행

```
kmeans = KMeans(n_clusters=3, init='k-means++', max_iter=300, random_state=0)
kmeans.fit(irisDF)
```

군집 레이블 확인

```
print(kmeans.labels_)
```

- 출력 결과: `[1 1 1 1 ... 2 0 2 0 2 2 ...]`
- `labels_` 값이 0, 1, 2로 구성 → 각 레코드가 첫 번째, 두 번째, 세 번째 군집에 속함을 의미

실제 분류 값과 군집화 결과 비교

```
irisDF['target'] = iris.target
irisDF['cluster'] = kmeans.labels_

iris_result = irisDF.groupby(['target', 'cluster'])['sepal_length'].count()
print(iris_result)
```

출력 결과

target	cluster	개수
0	1	50
1	0	48
1	2	2

target	cluster	개수
2	0	14
2	2	36

- 분류 타겟 0 → 1번 군집으로 모두 그룹핑
- 타겟 1 → 2개만 2번 군집, 나머지 48개는 0번 군집
- 타겟 2 → 0번 군집 14개, 2번 군집 36개로 분산

PCA를 이용한 2차원 시각화

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
pca_transformed = pca.fit_transform(iris.data)

irisDF['pca_x'] = pca_transformed[:, 0]
irisDF['pca_y'] = pca_transformed[:, 1]
irisDF.head(3)
```

출력 결과

	sepal_length	sepal_width	petal_length	petal_width	target	cluster	pca_x	pca_y
0	5.1	3.5	1.4	0.2	0	1	-2.684126	0.319397
1	4.9	3.0	1.4	0.2	0	1	-2.714142	-0.177001
2	4.7	3.2	1.3	0.2	0	1	-2.888991	-0.144949

산점도 시각화 코드

```
# 군집 값이 0, 1, 2인 경우마다 별도의 인덱스로 추출
marker0_ind = irisDF[irisDF['cluster'] == 0].index
marker1_ind = irisDF[irisDF['cluster'] == 1].index
marker2_ind = irisDF[irisDF['cluster'] == 2].index

# 군집 0, 1, 2에 해당하는 인덱스로 군집 레벨의 pca_x, pca_y 추출. o, s, ^ 마커 표시
plt.scatter(x=irisDF.loc[marker0_ind, 'pca_x'], y=irisDF.loc[marker0_ind, 'pca_y'], marker='o')
plt.scatter(x=irisDF.loc[marker1_ind, 'pca_x'], y=irisDF.loc[marker1_ind, 'pca_y'], marker='s')
plt.scatter(x=irisDF.loc[marker2_ind, 'pca_x'], y=irisDF.loc[marker2_ind, 'pca_y'], marker='^')

plt.xlabel('PCA 1')
plt.ylabel('PCA 2')
plt.title('3 Clusters Visualization by 2 PCA Components')
plt.show()
```

- Cluster 1(네모 's') → 다른 군집과 명확히 분리
- Cluster 0(둥그라미 'o')과 Cluster 2(세모 '^') → 상당 수준 분리되어 있으나 네모만큼 명확하지 않음
- Cluster 0과 1의 경우 속성의 위치 자체가 명확히 분리되기 어려운 부분 존재

군집화 알고리즘 테스트를 위한 데이터 생성

사이킷런 제공 데이터 생성 API

API	특징
<code>make_blobs()</code>	개별 군집의 중심점과 표준편차 제어 가능 추가. 군집화 테스트에 가장 많이 사용
<code>make_classification()</code>	노이즈를 포함한 데이터 생성에 유용. 분류 용도로도 사용 가능
<code>make_circle()</code>	중심 기반 군집화로 해결하기 어려운 데이터 세트 생성
<code>make_moon()</code>	중심 기반 군집화로 해결하기 어려운 데이터 세트 생성

`make_blobs()` 주요 파라미터

파라미터	설명
<code>n_samples</code>	생성할 총 데이터의 개수. 디폴트 100개
<code>n_features</code>	데이터의 피쳐 개수. 시각화 목표 시 2개 설정 권장 (x, y 좌표 표현)
<code>centers</code>	int 입력 시 군집 개수, ndarray 입력 시 개별 중심점의 좌표 의미
<code>cluster_std</code>	생성될 군집 데이터의 표준편차. 리스트로 군집별 다른 표준편차 설정 가능

`make_blobs()` 사용 예시

```
import numpy as np
import matplotlib.pyplot as plt
```

```

from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
%matplotlib inline

X, y = make_blobs(n_samples=200, n_features=2, centers=3,
                  cluster_std=0.8, random_state=0)

print(X.shape, y.shape)

# y target 값의 분포 확인
unique, counts = np.unique(y, return_counts=True)
print(unique, counts)

```

출력 결과

```

(200, 2) (200,)
[0 1 2] [67 67 66]

```

- X: 200개 레코드, 2개 피처 → shape (200, 2)
- y: shape (200,), 3개의 cluster 값 [0, 1, 2]이며 각각 67, 67, 66개로 균일 구성

DataFrame으로 변환 및 시각화

```

import pandas as pd

clusterDF = pd.DataFrame(data=X, columns=['ftr1', 'ftr2'])
clusterDF['target'] = y
clusterDF.head(3)

```

출력 결과

	ftr1	ftr2	target
0	-1.692427	3.622025	2
1	0.697940	4.428867	0
2	1.100228	4.606317	0

```

target_list = np.unique(y)

# 각 타깃별 산점도의 마커 값
markers = ['o', 's', '^', 'P', 'D', 'H', 'x']

for target in target_list:
    target_cluster = clusterDF[clusterDF['target'] == target]
    plt.scatter(x=target_cluster['ftr1'], y=target_cluster['ftr2'],
                edgecolor='k', marker=markers[target])

plt.show()

```

KMeans 군집화 수행 및 시각화

```

# KMeans 객체를 이용해 X 데이터를 K-Means 클러스터링 수행
kmeans = KMeans(n_clusters=3, init='k-means++', max_iter=200, random_state=0)
cluster_labels = kmeans.fit_predict(X)
clusterDF['kmeans_label'] = cluster_labels

# cluster_centers는 개별 클러스터의 중심 위치 좌표 시각화를 위해 추출
centers = kmeans.cluster_centers_
unique_labels = np.unique(cluster_labels)

```

```

markers = ['o', 's', '^', 'P', 'D', 'H', 'x']

# 군집된 label 유형별로 iteration 하면서 marker별로 scatter plot 수행
for label in unique_labels:
    label_cluster = clusterDF[clusterDF['kmeans_label'] == label]
    center_x_y = centers[label]
    plt.scatter(x=label_cluster['ftr1'], y=label_cluster['ftr2'],
                edgecolor='k', marker=markers[label])

# 군집별 중심 위치 좌표 시각화

```

```
plt.scatter(x=center_x_y[0], y=center_x_y[1], s=200, color='white',
            alpha=0.9, edgecolor='k', marker=markers[label])
plt.scatter(x=center_x_y[0], y=center_x_y[1], s=70, color='k',
            edgecolor='k', marker='<math>d</math>' % label)

plt.show()
```

target 값과 kmeans_label 비교

```
print(clusterDF.groupby('target')['kmeans_label'].value_counts())
```

출력 결과

target	kmeans_label	개수
0	0	66
0	1	1
1	2	67
2	1	65
2	2	1

- target 0 → cluster label 0, target 1 → label 2, target 2 → label 1로 거의 대부분 매핑

cluster_std에 따른 데이터 분포 변화

- cluster_std가 작을수록 → 군집 중심에 데이터가 모여 있음
- cluster_std가 클수록 → 데이터가 퍼져 있음

02. 군집 평가 (Cluster Evaluation)

군집 평가의 필요성

- 대부분의 군집화 데이터 세트는 비교할 만한 타깃 레이블을 가지고 있지 않음
- 군집화는 분류(Classification)와 유사해 보이나 성격이 많이 다름
 - 데이터 내에 숨어 있는 별도의 그룹을 찾아 의미 부여
 - 동일한 분류 값에 속하더라도 그 안에서 세분화된 군집화 추구
 - 서로 다른 분류 값의 데이터도 넓은 군집화 레벨화 등
- 비지도학습의 특성상 어떠한 지표라도 정확하게 성능 평가하기 어려움
- 대표적인 군집화 성능 평가 방법: 실루엣 분석(Silhouette Analysis)

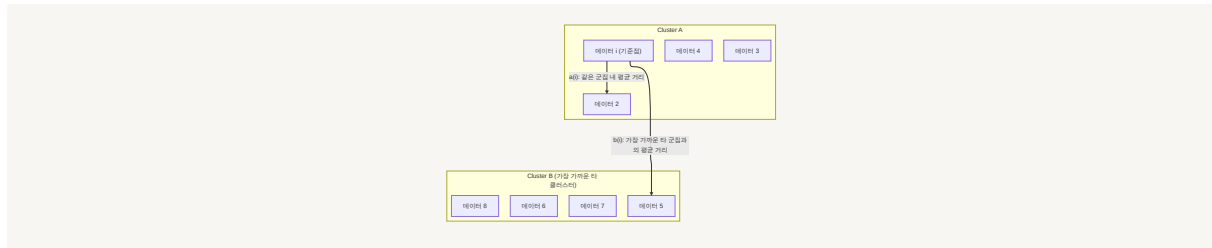
실루엣 분석 개요

- 각 군집 간의 거리가 얼마나 효율적으로 분리되어 있는지 나타냄
- 효율적 분리 = 다른 군집과의 거리는 떨어져 있고, 동일 군집끼리는 서로 가깝게 뭉쳐 있음
- 군집화가 잘 될수록 개별 군집은 비슷한 정도의 여유공간을 가지고 떨어져 있음

실루엣 계수(Silhouette Coefficient)

- 개별 데이터가 가지는 군집화 지표
- 해당 데이터가 같은 군집 내의 데이터와 얼마나 가깝게 군집화되어 있고, 다른 군집에 있는 데이터와는 얼마나 멀리 분리되어 있는지 나타내는 지표

```
graph TD
    subgraph ClusterA["Cluster A"]
        i["데이터 i (기준점)"]
        a2["데이터 2"]
        a3["데이터 3"]
        a4["데이터 4"]
    end
    subgraph ClusterB["Cluster B (가장 가까운 타 클러스터)"]
        b5["데이터 5"]
        b6["데이터 6"]
        b7["데이터 7"]
        b8["데이터 8"]
    end
    i -- "a(i): 같은 군집 내 평균 거리" --> a2
    i -- "b(i): 가장 가까운 타 군집과의 평균 거리" --> b5
```



실루엣 계수 계산 공식

- $a(i)$: i 번째 데이터에서 자신이 속한 클러스터 내의 다른 데이터 포인트들의 평균 거리
- $b(i)$: i 번째 데이터에서 가장 가까운 타 클러스터의 데이터 포인트들의 평균 거리
- 두 군집 간의 거리 차이: $b(i) - a(i)$
- 정규화: $\frac{b(i) - a(i)}{\max(a(i), b(i))}$ 로 나눔

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

실루엣 계수 해석

값 범위	의미
1에 가까울수록	군치의 군집과 멀리 떨어져 있음 (좋은 군집화)
0에 가까울수록	군치의 군집과 가까워짐
음수(-)	아예 다른 군집에 데이터 포인트가 할당됨

사이킷런의 실루엣 분석 API

API	설명
<code>silhouette_samples(X, labels, metric='euclidean')</code>	각 데이터 포인트의 실루엣 계수를 배열로 반환
<code>silhouette_score(X, labels, metric='euclidean')</code>	전체 데이터의 실루엣 계수 평균값 반환. <code>np.mean(silhouette_samples())</code> 와 동일

좋은 군집화 기준 조건

1. 전체 실루엣 계수의 평균값 (`silhouette_score()`) → 0~1 사이, 1에 가까울수록 좋음
2. 전체 실루엣 계수의 평균값과 더불어 개별 군집의 평균값의 편차가 크지 않아야 함
 - 개별 군집의 실루엣 계수 평균값이 전체 평균값에서 크게 벗어나지 않는 것이 중요
 - 전체 평균은 높지만 특정 군집만 유난히 높고 다른 군집들이 낮으면 좋은 군집화 아님

붓꽃 데이터 세트를 이용한 군집 평가

```

from sklearn.preprocessing import scale
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
# 실루엣 분석 평가 지표 값을 구하기 위한 API 추가
from sklearn.metrics import silhouette_samples, silhouette_score
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
%matplotlib inline

iris = load_iris()
feature_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
irisDF = pd.DataFrame(data=iris.data, columns=feature_names)

kmeans = KMeans(n_clusters=3, init='k-means++', max_iter=300, random_state=0).fit(irisDF)
irisDF['cluster'] = kmeans.labels_

# iris의 모든 개별 데이터에 실루엣 계수 값을 구함
score_samples = silhouette_samples(iris.data, irisDF['cluster'])
print('silhouette_samples() return 값의 shape', score_samples.shape)

# irisDF에 실루엣 계수 칼럼 추가
irisDF['silhouette_coeff'] = score_samples

# 모든 데이터의 평균 실루엣 계수 값을 구함
average_score = silhouette_score(iris.data, irisDF['cluster'])
print('붓꽃 데이터 세트 Silhouette Analysis Score: {0:.3f}'.format(average_score))

```

```
irisDF.head(3)
```

출력 결과

```
silhouette_samples() return 값의 shape (150,)
붓꽃 데이터 세트 Silhouette Analysis Score: 0.553
```

	sepal_length	sepal_width	petal_length	petal_width	cluster	silhouette_coeff
0	5.1	3.5	1.4	0.2	1	0.852955
1	4.9	3.0	1.4	0.2	1	0.815495
2	4.7	3.2	1.3	0.2	1	0.829315

- 평균 실루엣 계수: 0.553
- 1번 군집: 실루엣 계수 평균 약 0.8 (높은 수치)
- 다른 군집들의 실루엣 계수가 낮아 전체 평균이 0.553

군집별 평균 실루엣 계수 확인

```
irisDF.groupby('cluster')['silhouette_coeff'].mean()
```

출력 결과

cluster	silhouette_coeff 평균
0	0.417320
1	0.798140
2	0.451105

- 1번 군집: 0.79 (높음)
- 0번: 0.41, 2번: 0.45 → 1번에 비해 낮은 수치

군집별 평균 실루엣 계수의 시각화를 통한 군집 개수 최적화 방법

- 전체 데이터의 평균 실루엣 계수 값이 높다고 해서 반드시 최적의 군집 개수 아님
 - 특정 군집 내의 실루엣 계수 값만 너무 높고, 다른 군집은 내부 데이터끼리의 거리가 멀어 실루엣 계수 값이 낮아도 평균적으로 높은 값을 가질 수 있음
 - 개별 군집별로 적당히 분리된 거리를 유지하면서도 군집 내의 데이터가 뭉쳐 있는 경우가 최적

군집 개수별 실루엣 분석 비교

```
# make_blobs를 통해 4개의 군집 중심의 500개 2차원 데이터 세트 생성
from sklearn.datasets import make_blobs

X, y = make_blobs(n_samples=500, n_features=2, centers=4, cluster_std=1,
                  center_box=(-10.0, 10.0), shuffle=True, random_state=1)

# 군집 개수가 2, 3, 4, 5개일 때의 군집별 실루엣 계수 평균값을 시각화
visualize_silhouette([2, 3, 4, 5], X)
```

군집 개수 2 vs 3 vs 4 비교 결과

군집 개수	평균 실루엣 계수	특이 사항
2	0.704	높지만 0번 군집의 데이터가 내부에서 많이 떨어져짐
3	0.588	0번 군집 모두 평균보다 낮음
4	0.651	개별 군집의 평균값이 비교적 균일 → 가장 이상적

- 4개의 군집일 때가 가장 이상적인 군집화 개수 (군집별 실루엣 계수가 균일)

붓꽃 데이터 세트 최적 군집 개수 탐색

```
from sklearn.datasets import load_iris

iris = load_iris()
visualize_silhouette([2, 3, 4, 5], iris.data)
```

군집 개수	평균 실루엣 계수	비고
2	0.681	가장 높음
3	0.553	군집 간 실루엣 계수 편차 큼
4	0.498	더 낮아짐

군집 개수	평균 실루엣 계수	비고
5	0.493	더 낮아짐

- 붓꽃 데이터를 K-평균으로 군집화할 경우 **군집 개수를 2개로 하는 것이 가장 좋음**
- 실루엣 계수를 통한 K-평균 군집 평가 방법의 단점: 데이터양이 늘어나면 수행 시간 크게 증가, 몇 만 건 이상의 데이터에서 개인용 PC 수행 시 메모리 부족 에러 발생 가능

03. 평균 이동 (Mean Shift)

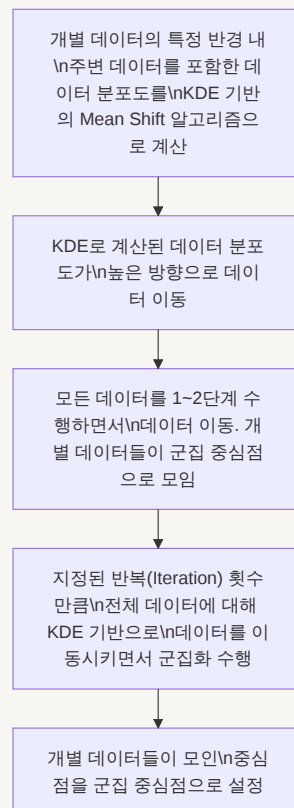
평균 이동(Mean Shift) 개요

flowchart TD

```

A["개별 데이터의 특정 반경 내\n주변 데이터를 포함한 데이터 분포도를\nKDE 기반의 Mean Shift 알고리즘으로 계산"] --> B["KDE로 계산된 데이터 분포도가\n높은 방향으로 데이터 이동"]
B --> C["모든 데이터를 1~2단계 수행하면서\n데이터 이동. 개별 데이터들이 군집 중심점으로 모임"]
C --> D["지정된 반복(Iteration) 횟수만큼\n전체 데이터에 대해 KDE 기반으로\n데이터를 이동시키면서 군집화 수행"]
D --> E["개별 데이터들이 모인\n중심점을 군집 중심점으로 설정"]

```



- K-평균과 유사하게 중심을 군집의 중심으로 지속적으로 움직이면서 군집화 수행
- **K-평균과의 차이:** K-평균이 중심에 소속된 데이터의 평균 거리 중심으로 이동하는 반면, **평균 이동은 중심을 데이터가 모여 있는 밀도가 가장 높은 곳으로 이동**
- 군집화를 위해 확률 밀도 함수(probability density function) 이용
- 가장 집중적으로 데이터가 모여 확률 밀도 함수가 피크(peak)인 점을 군집 중심점으로 선정
- 일반적으로 KDE(Kernel Density Estimation)를 이용해 확률 밀도 함수 추정

KDE (Kernel Density Estimation)

- 커널(Kernel) 함수를 통해 어떤 변수의 확률 밀도 함수를 추정하는 대표적인 방법
- 관측된 데이터 각각에 커널 함수를 적용한 값을 모두 더한 뒤 데이터 건수로 나눠 확률 밀도 함수를 추정
- 대표적인 커널 함수: **가우시안 분포 함수**

확률 밀도 함수(PDF, Probability Density Function)

- 확률 변수의 분포를 나타내는 함수
- 정규 분포, 감마 분포, t-분포 등
- 확률 밀도 함수를 알면 특정 변수가 어떤 값을 갖게 될지에 대한 확률 파악 가능

KDE 수식

$$KDE = \frac{1}{n} \sum_{i=1}^n K_h(x - x_i) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

기호	의미
K	커널 함수
x	확률 변수값
x _i	관측값
h	대역폭(bandwidth)

대역폭(bandwidth)의 중요성

- 대역폭 h: KDE 형태를 부드러운(또는 뾰족한) 형태로 평활화(Smoothing)하는 데 적용
- h 설정에 따라 확률 밀도 추정 성능이 크게 좌우됨

대역폭 h	결과
작은 h (예: h=1.0)	좁고 뾰족한 KDE → 변동성이 큰 방식으로 추정 → 과적합(over-fitting)
매우 큰 h (예: h=10)	과도하게 평활화된 KDE → 지나치게 단순화 → 과소적합(under-fitting)

- 일반적으로 평균 이동 군집화:
 - 대역폭이 클수록 → 평활화된 KDE로 인해 적은 수의 군집 중심점
 - 대역폭이 적을수록 → 많은 수의 군집 중심점
- 군집의 개수를 지정하지 않으며 오직 대역폭의 크기에 따라 군집화 수행

사이킷런 MeanShift 클래스

- `MeanShift` 클래스 제공
- 가장 중요한 초기화 파라미터: `bandwidth` (KDE의 대역폭 h와 동일)
- 최적의 대역폭 계산을 위해 `estimate_bandwidth()` 함수 제공

예제: `bandwidth=0.8` 적용

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import MeanShift

X, y = make_blobs(n_samples=200, n_features=2, centers=3, cluster_std=0.7, random_state=0)

meanshift = MeanShift(bandwidth=0.8)
cluster_labels = meanshift.fit_predict(X)
print('cluster labels 유형:', np.unique(cluster_labels))
```

출력 결과

```
cluster labels 유형: [0 1 2 3 4 5]
```

- 군집이 6개로 지나치게 세분화 → `bandwidth` 값이 너무 작음

`bandwidth=1.0`으로 조정

```
meanshift = MeanShift(bandwidth=1)
cluster_labels = meanshift.fit_predict(X)
print('cluster labels 유형:', np.unique(cluster_labels))
```

출력 결과

```
cluster labels 유형: [0 1 2]
```

- 3개의 군집으로 적절하게 군집화

`estimate_bandwidth()`로 최적 대역폭 계산

```
from sklearn.cluster import estimate_bandwidth

bandwidth = estimate_bandwidth(X)
print('bandwidth 값:', round(bandwidth, 3))
```

출력 결과

```
bandwidth 값: 1.816
```

estimate_bandwidth() 적용 군집화

```
import pandas as pd

clusterDF = pd.DataFrame(data=X, columns=['ftr1', 'ftr2'])
clusterDF['target'] = y

# estimate_bandwidth()로 최적의 bandwidth 계산
best_bandwidth = estimate_bandwidth(X)

meanshift = MeanShift(bandwidth=best_bandwidth)
cluster_labels = meanshift.fit_predict(X)
print('cluster labels 유형:', np.unique(cluster_labels))
```

출력 결과

```
cluster labels 유형: [0 1 2]
```

평균 이동 군집화 시각화

```
import matplotlib.pyplot as plt
%matplotlib inline

clusterDF['meanshift_label'] = cluster_labels
centers = meanshift.cluster_centers_
unique_labels = np.unique(cluster_labels)
markers = ['o', 's', '^', 'x', '*']

for label in unique_labels:
    label_cluster = clusterDF[clusterDF['meanshift_label'] == label]
    center_x_y = centers[label]

    # 군집별로 다른 마커로 산점도 적용
    plt.scatter(x=label_cluster['ftr1'], y=label_cluster['ftr2'],
                edgecolor='k', marker=markers[label])

    # 군집별 중심 표현
    plt.scatter(x=center_x_y[0], y=center_x_y[1], s=200, color='gray',
                alpha=0.9, marker=markers[label])
    plt.scatter(x=center_x_y[0], y=center_x_y[1], s=70, color='k',
                edgecolor='k', marker='$d$' % label)

plt.show()
```

target 값과 군집 label 비교

```
print(clusterDF.groupby('target')['meanshift_label'].value_counts())
```

출력 결과

target	meanshift_label	개수
0	0	67
1	1	67
2	2	66

- Target 값과 군집 label 값이 1:1로 완벽하게 매핑

평균 이동의 장단점

장점

- 데이터 세트의 형태를 특정 형태로 가정하지 않음
- 특정 분포도 기반의 모델로 가정하지 않아 유연한 군집화 가능
- 이상치의 영향력이 크지 않음
- 미리 군집의 개수를 정할 필요 없음

단점

- 알고리즘의 수행 시간이 오래 걸림
- bandwidth 크기에 따른 군집화 영향도가 매우 큼

주요 사용 분야

- 분석 업무 기반의 데이터 세트보다 **컴퓨터 비전 영역**에서 주로 사용
- 이미지나 영상 데이터에서 특정 개체를 구분하거나 움직임을 추적하는 데 뛰어난 성능

04. GMM (Gaussian Mixture Model)

GMM 소개

- 군집화를 적용하고자 하는 데이터가 여러 개의 ****가우시안 분포(Gaussian Distribution)****를 가진 데이터 집합들이 섞여서 생성된 것이라는 가정하에 군집화를 수행하는 방식
- 정규 분포(Normal Distribution): 좌우 대칭형의 종(Bell) 형태, 통계학에서 가장 잘 알려진 연속 확률 함수

정규 분포 특성

범위	데이터 포함 비율
평균 ± 표준편차 1 ($\pm 1\sigma$)	68.27%
평균 ± 표준편차 2 ($\pm 2\sigma$)	95.45%
평균 ± 표준편차 3 ($\pm 3\sigma$)	99.73%

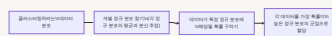
- 평균이 0이고 표준편차가 1인 정규 분포 → **표준 정규 분포**

[📷 이미지 삽입 권장: 정규 분포 종 모양 곡선 ($\pm 1\sigma$, $\pm 2\sigma$, $\pm 3\sigma$ 구간별 비율 68.27%, 95.45%, 99.73% 표시)]

GMM 동작 방식

flowchart LR

```
A["클러스터링하려는 데이터 분포"] --> B["개별 정규 분포 찾기 (각 정규 분포의 평균과 분산 추정)"]
B --> C["데이터가 특정 정규 분포에 해당될 확률 구하기"]
C --> D["각 데이터를 가장 확률이 높은 정규 분포의 군집으로 할당"]
```



- 데이터를 여러 개의 가우시안 분포가 섞인 것으로 간주
- 섞인 데이터 분포에서 개별 유형의 가우시안 분포를 추출

GMM 모수 추정

추정 대상 (모수 추정, 2가지)

- 개별 정규 분포의 평균과 분산
- 각 데이터가 어떤 정규 분포에 해당되는지의 확률

모수 추정 방법

- EM(Expectation and Maximization)** 방법 적용
- 사이킷런: `GaussianMixture` 클래스 제공 (`sklearn.mixture` 패키지에 위치)

GMM을 이용한 붓꽃 데이터 세트 군집화

데이터 로드

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
%matplotlib inline

iris = load_iris()
feature_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
irisDF = pd.DataFrame(data=iris.data, columns=feature_names)
irisDF['target'] = iris.target
```

GaussianMixture 주요 파라미터

파라미터	설명
<code>n_components</code>	Gaussian mixture 모델의 총 개수. K-평균의 <code>n_clusters</code> 와 같이 군집의 개수를 정하는 중요한 파라미터

GMM 군집화 수행

```
from sklearn.mixture import GaussianMixture

gmm = GaussianMixture(n_components=3, random_state=0).fit(iris.data)
gmm_cluster_labels = gmm.predict(iris.data)

# 군집화 결과를 irisDF의 'gmm_cluster' 칼럼명으로 저장
irisDF['gmm_cluster'] = gmm_cluster_labels
irisDF['target'] = iris.target

# target 값에 따라 gmm_cluster 값이 어떻게 매핑됐는지 확인
iris_result = irisDF.groupby(['target'])['gmm_cluster'].value_counts()
print(iris_result)
```

출력 결과 (GMM)

target	gmm_cluster	개수
0	0	50
1	2	45
1	1	5
2	1	50

K-평균과 비교

```
kmeans = KMeans(n_clusters=3, init='k-means++', max_iter=300, random_state=0).fit(iris.data)
kmeans_cluster_labels = kmeans.predict(iris.data)
irisDF['kmeans_cluster'] = kmeans_cluster_labels

iris_result = irisDF.groupby(['target'])['kmeans_cluster'].value_counts()
print(iris_result)
```

출력 결과 (K-평균)

target	kmeans_cluster	개수
0	1	50
1	0	48
1	2	2
2	2	36
2	0	14

비교 결론

- GMM: Target 0, 2는 완벽 매핑, Target 1은 45개(90%)만 2번 군집으로 매핑 → K-평균보다 효과적
- 이는 특정 알고리즘이 더 뛰어나다는 의미가 아니라 **붓꽃 데이터 세트가 GMM 군집화에 더 효과적**이라는 의미

GMM과 K-평균의 비교

K-평균의 특성

- 원형의 범위에서 군집화 수행
- 데이터 세트가 원형의 범위를 가질수록 군집화 효율 높아짐
- 타원형으로 늘어난 데이터에는 군집화를 잘 수행하지 못함

타원형 데이터 생성 및 비교

```
from sklearn.datasets import make_blobs

# make_blobs()로 300개의 데이터 세트, 3개의 군집 세트, cluster_std=0.5를 만들
X, y = make_blobs(n_samples=300, n_features=2, centers=3, cluster_std=0.5, random_state=0)

# 길게 늘어난 타원형의 데이터 세트를 생성하기 위해 변환
transformation = [[0.60834549, -0.63667341], [-0.40887718, 0.85253229]]
X_aniso = np.dot(X, transformation)

# feature 데이터 세트와 make_blobs()의 y 결과값을 DataFrame으로 저장
clusterDF = pd.DataFrame(data=X_aniso, columns=['ftr1', 'ftr2'])
```

```
clusterDF['target'] = y

# 생성된 데이터 세트를 target별로 다른 마커로 표시해 시각화
visualize_cluster_plot(None, clusterDF, 'target', iscenter=False)
```

KMeans 적용 (타원형 데이터)

```
# 3개의 군집 기반 Kmeans를 X_aniso 데이터 세트에 적용
kmeans = KMeans(3, random_state=0)
kmeans_label = kmeans.fit_predict(X_aniso)
clusterDF['kmeans_label'] = kmeans_label

visualize_cluster_plot(kmeans, clusterDF, 'kmeans_label', iscenter=True)
```

- KMeans: 원형 영역으로 군집화되어 원하는 방향으로 구성되지 않음

GMM 적용 (타원형 데이터)

```
# 3개의 n_components 기반 GMM을 X_aniso 데이터 세트에 적용
gmm = GaussianMixture(n_components=3, random_state=0)
gmm_label = gmm.fit(X_aniso).predict(X_aniso)
clusterDF['gmm_label'] = gmm_label

# GaussianMixture는 cluster_centers_ 속성이 없으므로 iscenter를 False로 설정
visualize_cluster_plot(gmm, clusterDF, 'gmm_label', iscenter=False)
```

- GMM: 데이터가 분포된 방향에 따라 정확하게 군집화

군집화 효율 차이 확인

```
print('### KMeans Clustering ###')
print(clusterDF.groupby('target')['kmeans_label'].value_counts())

print('\n### Gaussian Mixture Clustering ###')
print(clusterDF.groupby('target')['gmm_label'].value_counts())
```

출력 결과

```
### KMeans Clustering ###
target  kmeans_label
0       2            73
       0            27
1       1           100
2       0            86
       2            14

### Gaussian Mixture Clustering ###
target  gmm_label
0       2           100
1       1           100
2       0           100
```

- KMeans: 군집 1번만 정확히 매핑, 나머지 군집은 target 값과 어긋남
- GMM: 모든 군집이 target 값과 완벽 매핑

GMM의 특성

- KMeans보다 유연하게 다양한 데이터 세트에 적용 가능
- 단점: 군집화를 위한 수행 시간이 오래 걸림

05. DBSCAN

DBSCAN 개요

- **DBSCAN**: Density Based Spatial Clustering of Applications with Noise
- 밀도 기반 군집화의 대표적인 알고리즘
- 간단하고 직관적인 알고리즘임에도 **데이터의 분포가 기하학적으로 복잡한 데이터 세트**에도 효과적인 군집화 가능
- 특정 공간 내에 데이터 밀도 차이를 기반으로 군집화 수행
- K-평균, 평균 이동, GMM으로 해결하기 어려운 내부 원/외부 원 형태의 복잡한 기하학적 분포도를 가진 데이터에 효과적

DBSCAN 핵심 개념

주요 파라미터

파라미터	설명
입실론(epsilon) 주변 영역	개별 데이터를 중심으로 입실론 반경을 가지는 원형의 영역
최소 데이터 개수(min points)	개별 데이터의 입실론 주변 영역에 포함되는 타 데이터의 최소 개수

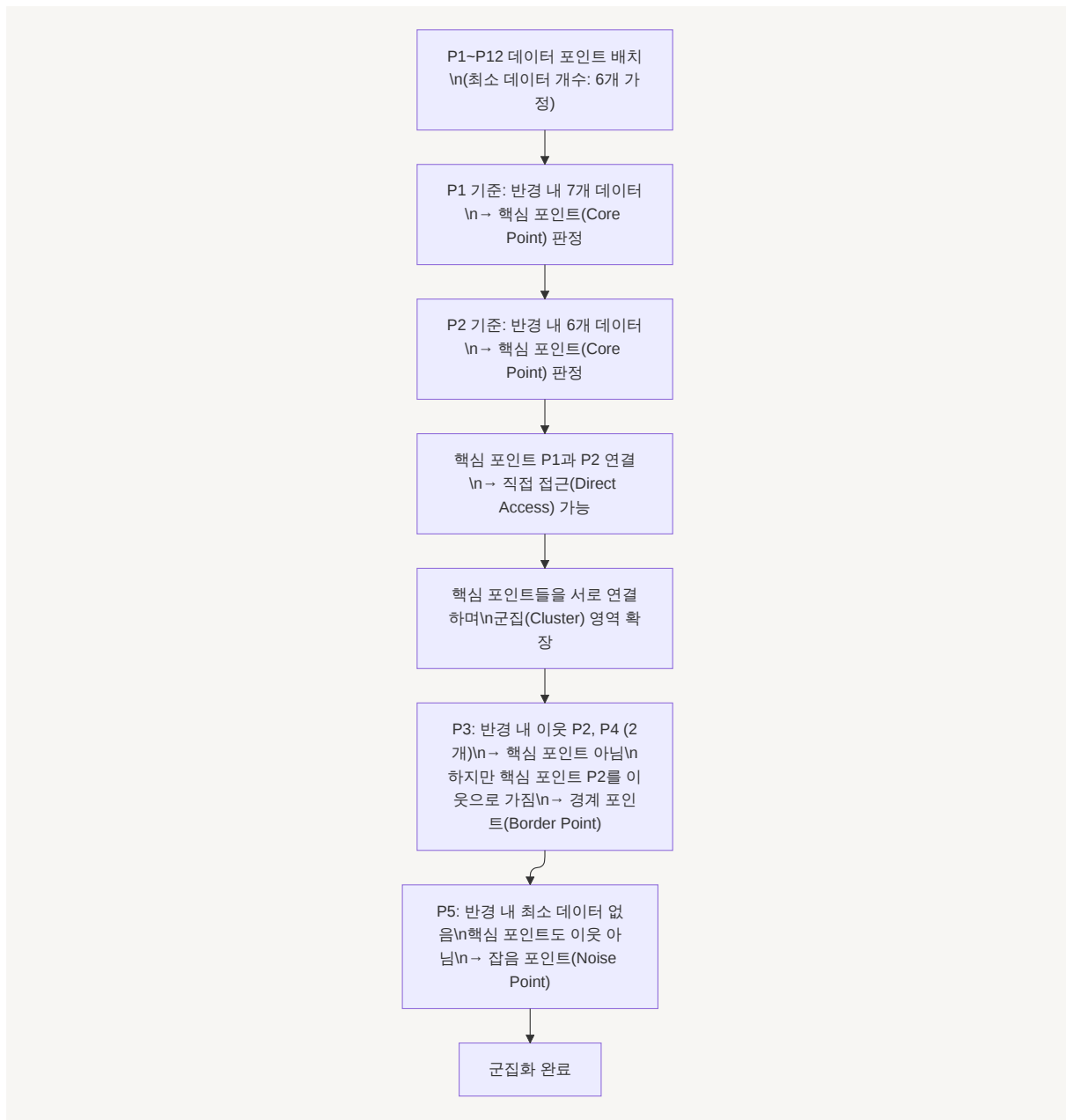
데이터 포인트 분류

포인트 종류	정의
핵심 포인트 (Core Point)	주변 영역 내에 최소 데이터 개수 이상의 타 데이터를 가지고 있는 데이터
이웃 포인트 (Neighbor Point)	주변 영역 내에 위치한 타 데이터
경계 포인트 (Border Point)	주변 영역 내에 최소 데이터 개수 이상의 이웃 포인트를 가지고 있지 않지만 핵심 포인트를 이웃 포인트로 가지고 있는 데이터 → 군집의 외곽 형성
잡음 포인트 (Noise Point)	최소 데이터 개수 이상의 이웃 포인트를 가지고 있지 않으며 핵심 포인트도 이웃 포인트로 가지고 있지 않는 데이터

DBSCAN 동작 원리 (단계별)

```

flowchart TD
    A["P1~P12 데이터 포인트 배치\\n(최소 데이터 개수: 6개 가정)"] --> B["P1 기준: 반경 내 7개 데이터\\n→ 핵심 포인트(Core Point) 판정"]
    B --> C["P2 기준: 반경 내 6개 데이터\\n→ 핵심 포인트(Core Point) 판정"]
    C --> D["핵심 포인트 P1과 P2 연결\\n→ 직접 접근(Direct Access) 가능"]
    D --> E["핵심 포인트들을 서로 연결하며\\n군집(Cluster) 영역 확장"]
    E --> F["P3: 반경 내 이웃 P2, P4 (2개)\\n→ 핵심 포인트 아님\\n하지만 핵심 포인트 P2를 이웃으로 가짐\\n→ 경계 포인트(Border Point)"]
    F --> G["P5: 반경 내 최소 데이터 없음\\n핵심 포인트도 이웃 아님\\n→ 잡음 포인트(Noise Point)"]
    G --> H["군집화 완료"]
  
```



단계 상세 설명

1. P1 데이터 기준 입실론 반경 내 7개 데이터(P1, P2, P6, P7, P8, P9, P11) → 최소 데이터 5개 이상 만족 → P1은 핵심 포인트
2. P2 역시 반경 내 6개 데이터(P2, P1, P3, P4, P9, P10) → 핵심 포인트
3. 핵심 포인트 P1의 이웃 P2도 핵심 포인트 → P1에서 P2로 직접 접근 가능
4. 특정 핵심 포인트에서 직접 접근 가능한 다른 핵심 포인트를 서로 연결하면서 군집 영역 확장 → DBSCAN 군집화 방식
5. P3: 반경 내 이웃 데이터 P2, P4로 2개 → 핵심 포인트 아님, 하지만 핵심 포인트 P2를 이웃으로 가짐 → **경계 포인트** (군집의 외곽 형성)
6. P5: 반경 내 최소 데이터 없고 핵심 포인트도 이웃 아님 → **잡음 포인트**
7. P12: 별도로 떨어진 위치 → 잡음 포인트

사이킷런 DBSCAN 클래스

```
from sklearn.cluster import DBSCAN
```

주요 초기화 파라미터

파라미터	설명
<code>eps</code>	입실론 주변 영역의 반경. 일반적으로 1 이하의 값 설정
<code>min_samples</code>	핵심 포인트가 되기 위해 입실론 주변 영역 내에 포함되어야 할 데이터의 최소 개수 (자신의 데이터 포함, min points + 1)

파라미터	설명
<code>metric</code>	거리 측정 방식. 기본값 'euclidean'

DBSCAN 적용하기 - 붓꽃 데이터 세트

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.6, min_samples=8, metric='euclidean')
dbscan_labels = dbscan.fit_predict(iris.data)

irisDF['dbscan_cluster'] = dbscan_labels
irisDF['target'] = iris.target

iris_result = irisDF.groupby(['target'])['dbscan_cluster'].value_counts()
print(iris_result)
```

출력 결과 (eps=0.6, min_samples=8)

target	dbscan_cluster	개수
0	0	49
0	-1	1
1	1	46
1	-1	4
2	1	42
2	-1	8

- `dbscan_cluster` 값 -1 → 노이즈에 속하는 군집 (잡음 포인트)
- 붓꽃 데이터는 DBSCAN에서 0과 1, 2개의 군집으로 군집화
- 군집 개수가 3개(target)인데 2개로 군집화됐다고 군집화 효율이 떨어진다는 의미가 아님
- DBSCAN은 군집의 개수를 알고리즘에 따라 자동으로 지정

PCA 2차원 변환 후 시각화

```
from sklearn.decomposition import PCA

# 2차원으로 시각화하기 위해 PCA n_components=2로 피쳐 데이터 세트 변환
pca = PCA(n_components=2, random_state=0)
pca_transformed = pca.fit_transform(iris.data)

# visualize_cluster_plot() 함수는 ftr1, ftr2 칼럼을 좌표에 표현하므로 PCA 변환값을 해당 칼럼으로 생성
irisDF['ftr1'] = pca_transformed[:, 0]
irisDF['ftr2'] = pca_transformed[:, 1]

visualize_cluster_plot(dbscan, irisDF, 'dbscan_cluster', iscenter=False)
```

- 별표(★): 노이즈 데이터
- PCA 2차원으로 표현 시 이상치인 노이즈 데이터 명확하게 드러남

DBSCAN 파라미터 변화에 따른 영향

eps 조정 (0.6 → 0.8)

```
dbscan = DBSCAN(eps=0.8, min_samples=8, metric='euclidean')
dbscan_labels = dbscan.fit_predict(iris.data)
```

출력 결과 (eps=0.8)

target	dbscan_cluster	개수
0	0	50
1	1	50
2	1	47
2	-1	3

- 노이즈가 3개로 대폭 감소 → eps 변경이 커지면서 기존 노이즈들이 Cluster 1에 소속

min_samples 조정 (8 → 16, eps=0.6 유지)


```
dbscan = DBSCAN(eps=0.6, min_samples=16, metric='euclidean')
```

출력 결과 (eps=0.6, min_samples=16)

target	dbscan_cluster	개수
0	0	48
0	-1	2
1	1	44
1	-1	6
2	1	36
2	-1	14

- 노이즈 데이터가 기존보다 많이 증가

파라미터 영향 정리

파라미터 변화	노이즈 데이터
eps 값 크게	노이즈 감소 (포함하는 데이터 증가)
eps 값 작게	노이즈 증가
min_samples 크게	노이즈 증가 (더 많은 데이터를 포함해야 핵심 포인트 인정)
min_samples 작게	노이즈 감소

DBSCAN 적용하기 - make_circles() 데이터 세트

복잡한 기하학적 분포 데이터 생성

```
from sklearn.datasets import make_circles

X, y = make_circles(n_samples=1000, shuffle=True, noise=0.05, random_state=0, factor=0.5)
clusterDF = pd.DataFrame(data=X, columns=['ftr1', 'ftr2'])
clusterDF['target'] = y

visualize_cluster_plot(None, clusterDF, 'target', iscenter=False)
```

make_circles() 파라미터

파라미터	설명
noise	노이즈 데이터 세트의 비율
factor	외부 원과 내부 원의 scale 비율

K-평균 적용 (make_circles 데이터)

```
# KMeans로 make_circles() 데이터 세트를 군집화 수행
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=2, max_iter=1000, random_state=0)
kmeans_labels = kmeans.fit_predict(X)
clusterDF['kmeans_cluster'] = kmeans_labels

visualize_cluster_plot(kmeans, clusterDF, 'kmeans_cluster', iscenter=True)
```

- 위/아래 군집 중심을 기반으로 위아래 절반으로 군집화 → 원하는 결과 아님
- 거리 기반 군집화로는 데이터가 특정 형태로 지속해서 이어지는 부분을 찾아내기 어려움

GMM 적용 (make_circles 데이터)

```
# GMM으로 make_circles() 데이터 세트를 군집화 수행
from sklearn.mixture import GaussianMixture

gmm = GaussianMixture(n_components=2, random_state=0)
gmm_label = gmm.fit(X).predict(X)
clusterDF['gmm_cluster'] = gmm_label

visualize_cluster_plot(gmm, clusterDF, 'gmm_cluster', iscenter=False)
```

- GMM도 내부/외부 원형으로 구성된 복잡한 형태의 데이터 세트에서는 군집화가 원하는 방향으로 되지 않음

DBSCAN 적용 (make_circles 데이터)

```
# DBSCAN으로 make_circles() 데이터 세트 군집화 수행
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.2, min_samples=10, metric='euclidean')
dbscan_labels = dbscan.fit_predict(X)
clusterDF['dbscan_cluster'] = dbscan_labels

visualize_cluster_plot(dbscan, clusterDF, 'dbscan_cluster', iscenter=False)
```

- DBSCAN: 원하는 방향으로 정확히 군집화 성공

알고리즘별 make_circles 데이터 군집화 비교 요약

알고리즘	make_circles 결과
KMeans	위아래 절반으로 군집화 (실패)
GMM	타원형 데이터엔 강하지만 원형 내/외 구분 실패
DBSCAN	내부 원/외부 원 정확히 구분 (성공)

06. 군집화 실습 - 고객 세그멘테이션

고객 세그멘테이션의 정의와 기법

- 고객 세그멘테이션(Customer Segmentation): 다양한 기준으로 고객을 분류하는 기법
- CRM이나 마케팅의 중요 기반 요소

고객 분류 요소

신상 데이터	구매 행동 데이터
사는 지역	선호 상품군
결혼 여부	월별 사용액
성별	VIP 상품 사용 여부
소득	최근 구매 상품
직업	구매 주기, 년간 프로모션 참가 횟수

- 고객 세그멘테이션의 주요 목표: 타겟 마케팅
 - 고객을 여러 특성에 맞게 세분화해서 유형에 따라 맞춤형 마케팅이나 서비스를 제공

RFM 기법

지표	의미
Recency (R)	가장 최근 상품 구입 일에서 오늘까지의 기간
Frequency (F)	상품 구매 횟수
Monetary Value (M)	총 구매 금액

데이터 세트 로딩과 데이터 클렌징

데이터 소스

- URL: <http://archive.ics.uci.edu/ml/datasets/online+retail>
- 파일: [Online Retail.xlsx](#)
- 영국 기반 온라인 판매 데이터 (2010/12/01 ~ 2011/12/09)

데이터 로드

```
import pandas as pd
import datetime
import math
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

retail_df = pd.read_excel(io='Online Retail.xlsx')
retail_df.head(3)
```

데이터 세트 칼럼 설명

칼럼명	설명
InvoiceNo	주문번호. 'C'로 시작하는 것은 취소 주문

컬럼명	설명
StockCode	제품 코드(Item Code)
Description	제품 설명
Quantity	주문 제품 건수
InvoiceDate	주문 일자
UnitPrice	제품 단가
CustomerID	고객 번호
Country	국가명(주문 고객의 국적)

데이터 기본 정보 확인

```
retail_df.info()
```

출력 결과

```
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
InvoiceNo      541909 non-null object
StockCode      541909 non-null object
Description     540455 non-null object
Quantity       541909 non-null int64
InvoiceDate    541909 non-null datetime64[ns]
UnitPrice      541909 non-null float64
CustomerID     406829 non-null float64
Country        541909 non-null object
```

- 전체 데이터: 541,909개
- CustomerID Null 값: 135,080건 (406,829개만 Not Null)

데이터 전처리 (클렌징)

처리 항목

1. **Null 데이터 제거:** CustomerID Null 데이터 삭제 (고객 세그먼트에이션에 고객 식별 번호 필수)
2. **오류 데이터 삭제:** Quantity 또는 UnitPrice가 0보다 작은 경우 (반환 데이터 포함) 모두 삭제

```
retail_df = retail_df[retail_df['Quantity'] > 0]
retail_df = retail_df[retail_df['UnitPrice'] > 0]
retail_df = retail_df[retail_df['CustomerID'].notnull()]
print(retail_df.shape)
retail_df.isnull().sum()
```

출력 결과

```
(397884, 8)
InvoiceNo      0
StockCode      0
Description     0
Quantity       0
InvoiceDate    0
UnitPrice      0
CustomerID     0
Country        0
```

국가별 주문 건수 확인 및 영국 데이터만 필터링

```
retail_df['Country'].value_counts()[:5]
```

출력 결과

국가	건수
United Kingdom	354321
Germany	9040
France	8341
EIRE	7236
Spain	2484

```
retail_df = retail_df[retail_df['Country'] == 'United Kingdom']
print(retail_df.shape)
```

출력 결과

(354321, 8)

RFM 기반 데이터 가공

주문 금액 칼럼 생성 및 CustomerID 타입 변환

```
retail_df['sale_amount'] = retail_df['Quantity'] * retail_df['UnitPrice']
retail_df['CustomerID'] = retail_df['CustomerID'].astype(int)
```

Top-5 주문 건수 및 주문 금액 확인

```
print(retail_df['CustomerID'].value_counts().head(5))
print(retail_df.groupby('CustomerID')['sale_amount'].sum().sort_values(ascending=False)[:5])
```

출력 결과

CustomerID	주문 건수
17841	7847
14096	5111
12748	4595
14606	2700
15311	2379

CustomerID	주문 금액
18102	259657.30
17450	194550.79
16446	168472.50
17511	91062.38
16029	81024.84

- 소매점의 대규모 주문 포함 → 특정 고객에게 주문 건수/금액 집중

CustomerID 기준 GroupBy + 다중 Aggregation (agg() 활용)

```
# DataFrame의 groupby()의 multiple 연산을 위해 agg() 이용
# Recency는 InvoiceDate 칼럼의 max()에서 데이터 가공
# Frequency는 InvoiceNo 칼럼의 count(), Monetary value는 sale_amount 칼럼의 sum()

aggregations = {
    'InvoiceDate': 'max',
    'InvoiceNo': 'count',
    'sale_amount': 'sum'
}

cust_df = retail_df.groupby('CustomerID').agg(aggregations)

# groupby된 결과 칼럼 값을 Recency, Frequency, Monetary로 변경
cust_df = cust_df.rename(columns={
    'InvoiceDate': 'Recency',
    'InvoiceNo': 'Frequency',
    'sale_amount': 'Monetary'
})

cust_df = cust_df.reset_index()
cust_df.head(3)
```

출력 결과

CustomerID	Recency	Frequency	Monetary
12346	2011-01-18 10:01:00	1	77183.60
12747	2011-12-07 14:34:00	103	4196.01
12748	2011-12-09 12:20:00	4595	33719.73

Recency 변환 (최근 주문 일자 → 경과 일수)

```
import datetime as dt

# 오늘 날짜: 데이터 최종 날짜(2011-12-09) + 1일 = 2011-12-10
```

```

cust_df['Recency'] = dt.datetime(2011, 12, 10) - cust_df['Recency']
cust_df['Recency'] = cust_df['Recency'].apply(lambda x: x.days + 1)

print('cust_df 로우와 칼럼 건수는', cust_df.shape)
cust_df.head(3)

```

출력 결과

cust_df 로우와 칼럼 건수는 (3920, 4)

CustomerID	Recency	Frequency	Monetary
12346	326	1	77183.60
12747	3	103	4196.01
12748	1	4595	33719.73

- 고객 수: 3,920명
- Recency: 현재 날짜(2011-12-10) 기준 가장 최근 주문으로부터 경과 일수

RFM 기반 고객 세그멘테이션

데이터 분포 확인 (히스토그램)

```

fig, (ax1, ax2, ax3) = plt.subplots(figsize=(12, 4), nrows=1, ncols=3)
ax1.set_title('Recency Histogram')
ax1.hist(cust_df['Recency'])

ax2.set_title('Frequency Histogram')
ax2.hist(cust_df['Frequency'])

ax3.set_title('Monetary Histogram')
ax3.hist(cust_df['Monetary'])

plt.show()

```

기술 통계 확인

```
cust_df[['Recency', 'Frequency', 'Monetary']].describe()
```

출력 결과

	Recency	Frequency	Monetary
count	3920.0	3920.0	3920.0
mean	92.74	90.39	1864.39
std	99.53	217.81	7482.82
min	1.0	1.0	3.75
25%	18.0	17.0	300.28
50%	51.0	41.0	652.28
75%	143.0	99.25	1576.59
max	374.0	7847.0	259657.3

- Recency: 평균 92.7, 중위값 51 → 왜곡 존재
- Frequency: 평균 90.3, 75% 99.25, max 7847 → 심한 왜곡
- Monetary: 평균 1864.3, 75% 1576.5, max 259657.3 → 심한 왜곡

StandardScaler 적용 후 K-평균 수행

```

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, silhouette_samples

X_features = cust_df[['Recency', 'Frequency', 'Monetary']].values
X_features_scaled = StandardScaler().fit_transform(X_features)

kmeans = KMeans(n_clusters=3, random_state=0)
labels = kmeans.fit_predict(X_features_scaled)
cust_df['cluster_label'] = labels

```

```
print('실루엣 스코어는: {0:.3f}'.format(silhouette_score(X_features_scaled, labels)))
```

출력 결과

실루엣 스코어는: 0.592

- 군집 3개 기준 평균 실루엣 스코어: 0.592 (안정적인 수치)
- 하지만 군집별 실루엣 계수 분포 확인 필요

```
visualize_silhouette([2, 3, 4, 5], X_features_scaled)
visualize_kmeans_plot_multi([2, 3, 4, 5], X_features_scaled)
```

문제점

- 군집 2개: 너무 개괄적으로 군집화
- 군집 3개 이상: 데이터 세트 개수가 너무 작은 군집이 만들어짐
- 소수의 데이터 세트 = 왜곡된 데이터 값인 특정 소매점의 대량 주문 구매 데이터
- 군집 수를 계속 늘려봐야 해당 군집만 지속적으로 분리 → 의미 없는 군집화 결과

로그 변환 적용 후 K-평균 수행

- 왜곡 정도를 낮추기 위해 가장 자주 사용되는 방법: **데이터 값에 로그(Log) 적용**

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, silhouette_samples

# Recency, Frequency, Monetary 칼럼에 np.log1p()로 Log Transformation
cust_df['Recency_log'] = np.log1p(cust_df['Recency'])
cust_df['Frequency_log'] = np.log1p(cust_df['Frequency'])
cust_df['Monetary_log'] = np.log1p(cust_df['Monetary'])

# Log Transformation 데이터에 StandardScaler 적용
X_features = cust_df[['Recency_log', 'Frequency_log', 'Monetary_log']].values
X_features_scaled = StandardScaler().fit_transform(X_features)

kmeans = KMeans(n_clusters=3, random_state=0)
labels = kmeans.fit_predict(X_features_scaled)

cust_df['cluster_label'] = labels

print('실루엣 스코어는: {0:.3f}'.format(silhouette_score(X_features_scaled, labels)))
```

출력 결과

실루엣 스코어는: 0.303

```
visualize_silhouette([2, 3, 4, 5], X_features_scaled)
visualize_kmeans_plot_multi([2, 3, 4, 5], X_features_scaled)
```

로그 변환 전후 비교

구분	실루엣 스코어	군집화 품질
StandardScaler만 적용	0.592	특정 소수 군집에 집중 → 불균형
로그 변환 + StandardScaler	0.303	스코어는 낮지만 군집이 더 균일하게 분리

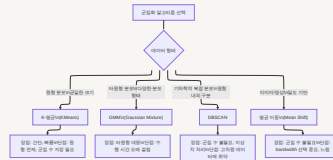
- 실루엣 스코어의 절대치보다 **개별 군집이 균일하게 나뉘는지**가 중요
- 왜곡된 데이터 세트에는 **로그 변환으로 일차 변환 후 군집화**를 수행하면 더 나은 결과 도출

전체 알고리즘 종합 비교

```
graph TD
    A["군집화 알고리즘 선택"] --> B["데이터 형태"]
    B -- "원형 분포\&#x2191; 균일한 크기" --> C["K-평균\&#x2191; (KMeans)"]
    B -- "타원형 분포\&#x2191; 다양한 분포 형태" --> D["GMM\&#x2191; (Gaussian Mixture)"]
```

B -- "기하학적 복잡 분포\\n원형 내외 구분" --> E["DBSCAN"]
 B -- "이미지/영상\\n밀도 기반" --> F["평균 이동\\n(Mean Shift)"]

C --> C1["장점: 간단, 빠름\\n단점: 원형 전제, 군집 수 지정 필요"]
 D --> D1["장점: 타원형 대응\\n단점: 수행 시간 오래 걸림"]
 E --> E1["장점: 군집 수 불필요, 이상치 처리\\n단점: 고차원 데이터에 취약"]
 F --> F1["장점: 군집 수 불필요\\n단점: bandwidth 선택 중요, 느림"]



최종 정리

알고리즘	군집 수 지정	거리/밀도	비선형 군집	주요 파라미터	수행 속도
K-평균	필요 (n_clusters)	거리 기반	불가	n_clusters, init, max_iter	빠름
평균 이동	불필요	밀도 기반	가능	bandwidth	느림
GMM	필요 (n_components)	확률 기반	부분 가능	n_components	느림
DBSCAN	불필요	밀도 기반	가능	eps, min_samples	중간